

Beyond the Agent Object

Entity Component Systems as an Architecture for Agent-
Based Modeling

Franz Scharnreitner

ICAE Linz

2026-09-09

1. Cars driving in circles

1.1 The original model

1. Cars driving in circles

- Influential paper by Hodgson & Knudsen (2006)

1.1 The original model

1. Cars driving in circles

- Influential paper by Hodgson & Knudsen (2006)



1.2 Overview

1. Cars driving in circles

- Cars move sequentially, either clockwise or counterclockwise, on a 100×2 ring.
- In each step, a car chooses the left or right lane according to the decision formula

$$\text{LR}^n = s^n + o^n + c^n + h^n$$

.

- Here, s^n , o^n , and c^n depend on the cars ahead of car n , while h^n represents the driver's habit.
- Driving on one lane shifts habit toward that lane, so $h^n = h_{\text{prev}}^n + \frac{\text{lane}}{K+t^n}$.

1.3 Main results of Hodgson & Knudsen

1. Cars driving in circles

- **Habit is crucial for the emergence of convention.**
- Repeated lane choices gradually become ingrained, making left/right coordination more stable.
- Agents do not coordinate only because of immediate incentives; they also coordinate because past behavior shapes current disposition.
- Habit therefore strengthens and stabilizes conventions beyond what purely reactive decision-making can achieve.
- The paper's broader claim is that institutions persist partly because they become internalized as habits.

1.4 The real world

1. Cars driving in circles

- Drivers on a roundabout decide on the basis of locally observable cues: available gaps, speeds, brake lights, and lateral positions.
- Decisions are taken continuously and concurrently, without a global clock or fixed turn order (Huberman & Glance, 1993).
- The intentions of other drivers remain unobserved until expressed as visible motion.

1.4 The real world

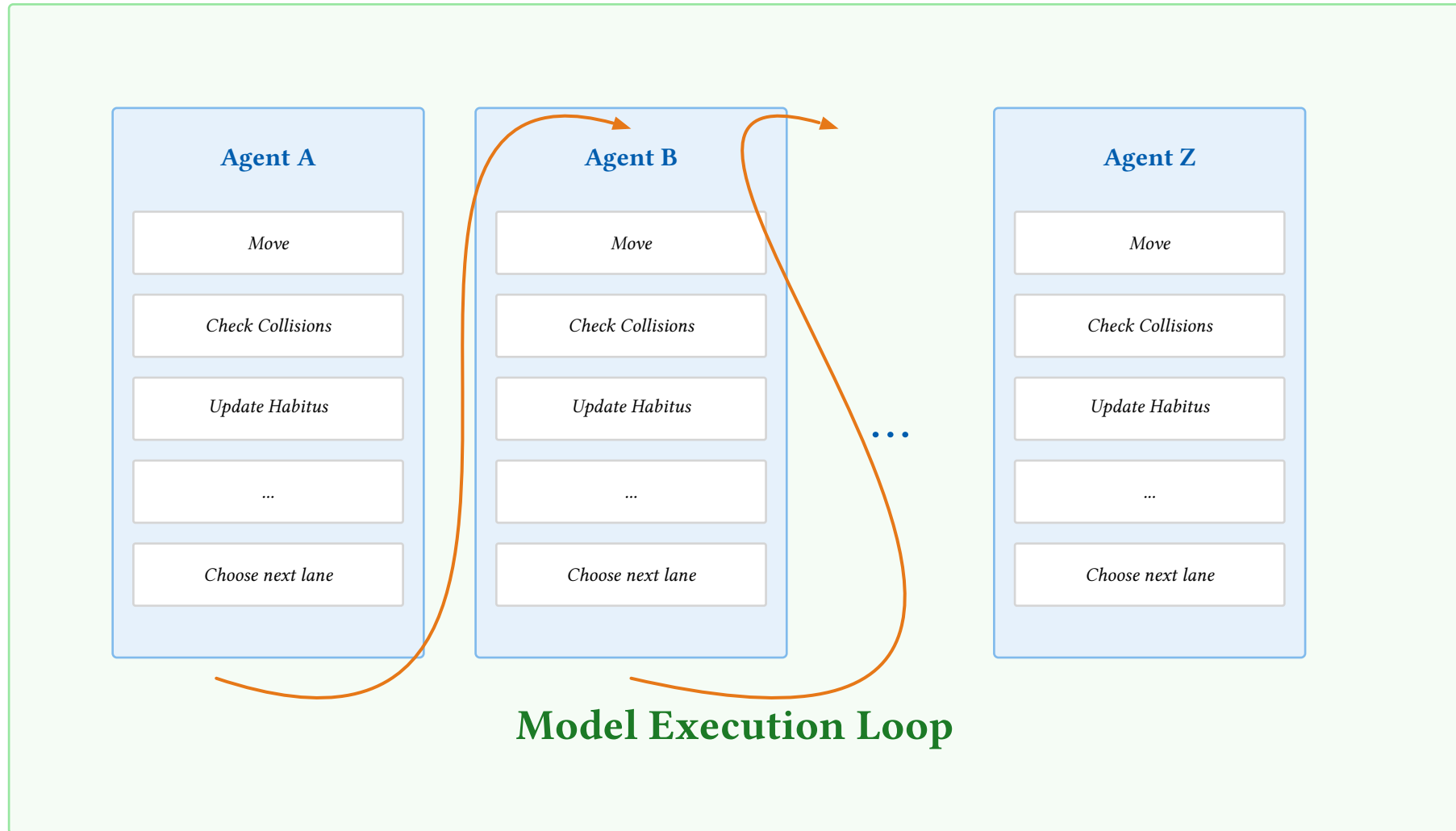
1. Cars driving in circles

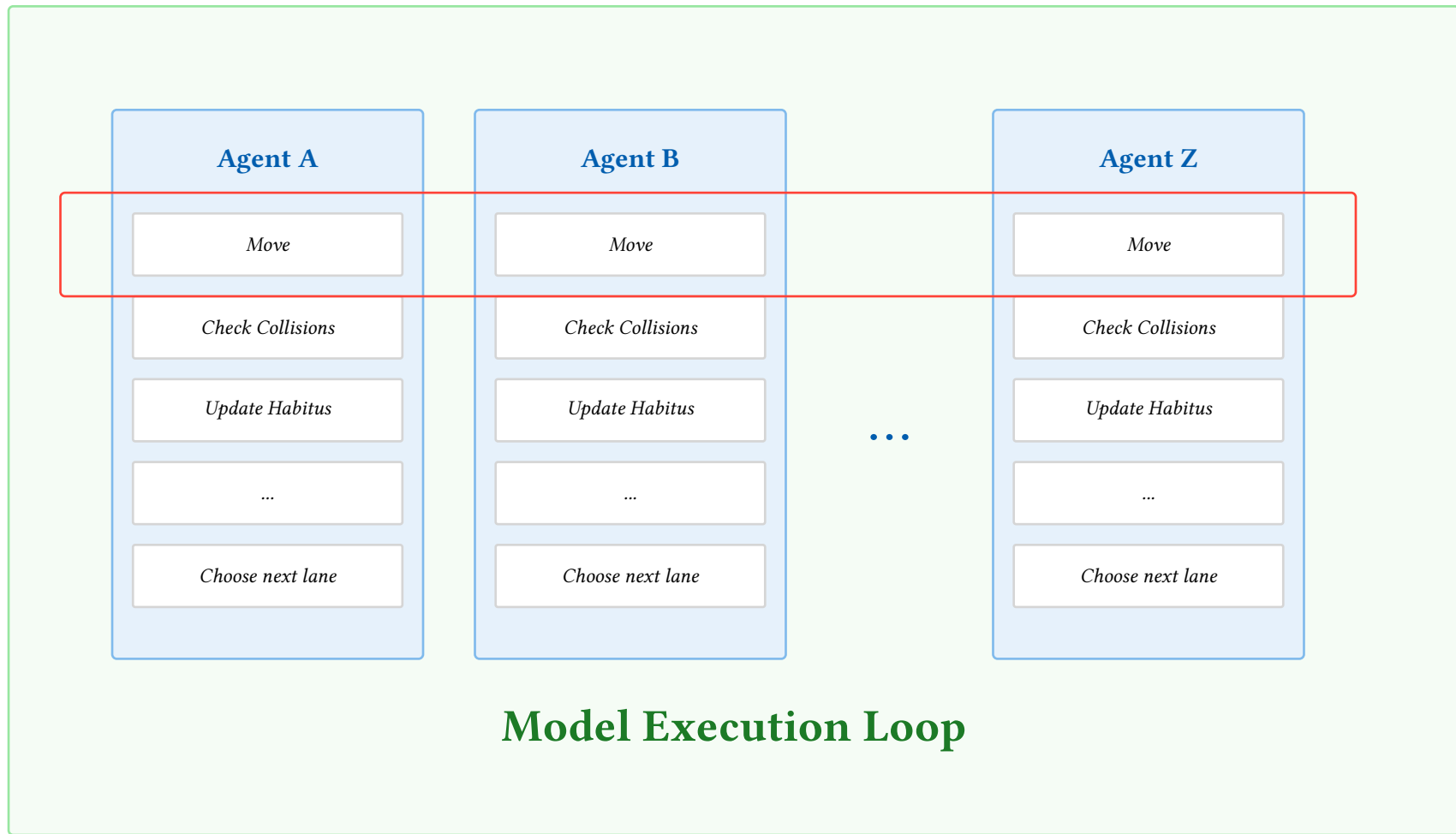
- Drivers on a roundabout decide on the basis of locally observable cues: available gaps, speeds, brake lights, and lateral positions.
- Decisions are taken continuously and concurrently, without a global clock or fixed turn order (Huberman & Glance, 1993).
- The intentions of other drivers remain unobserved until expressed as visible motion.
- To model this: all proposals from one observable state, then joint resolution
- What is the problem when modeling this in an “agent-centered framework”?

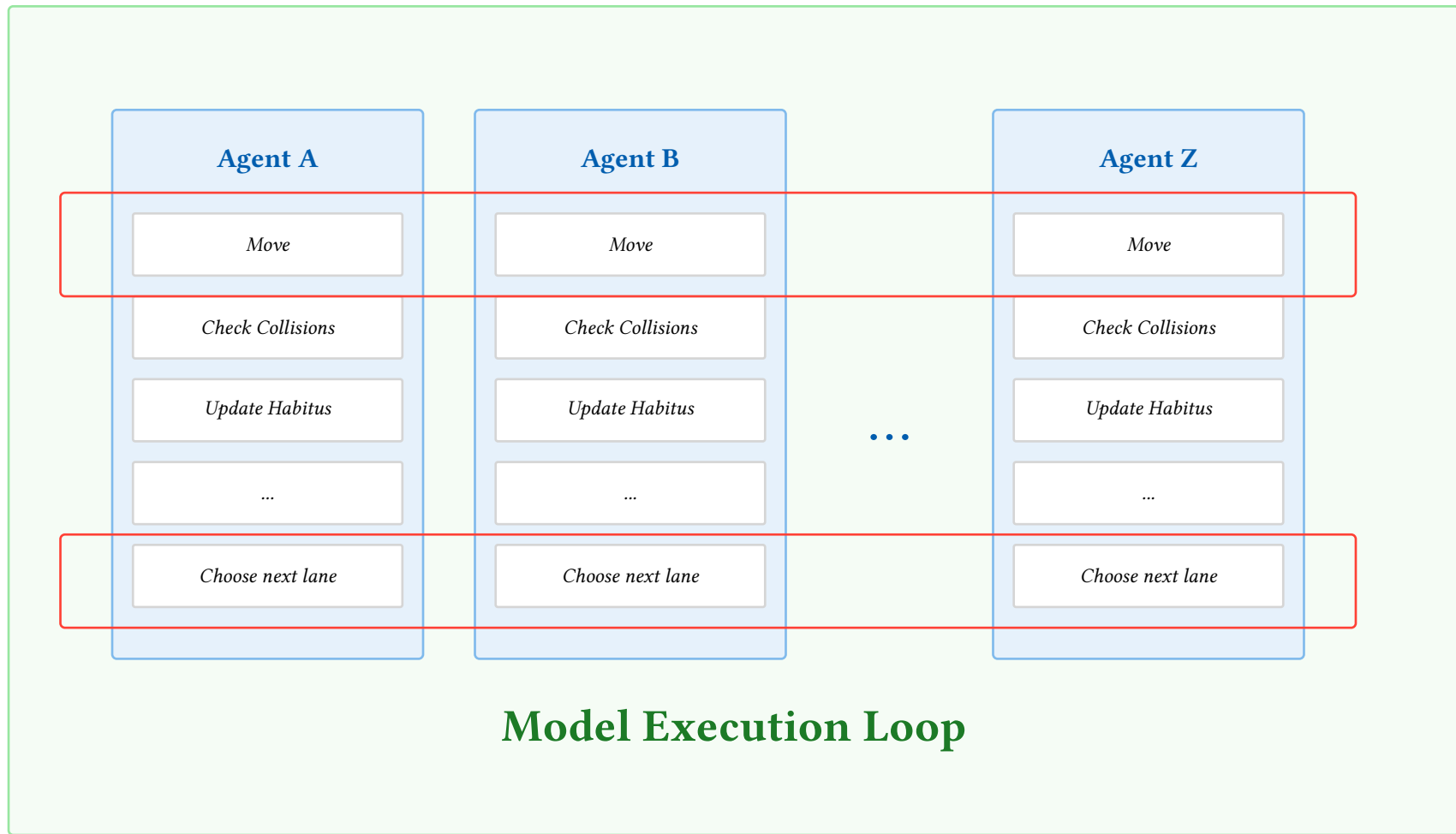
2. *ABM layouts*

2.1 Traditional ABM layout

2. ABM layouts





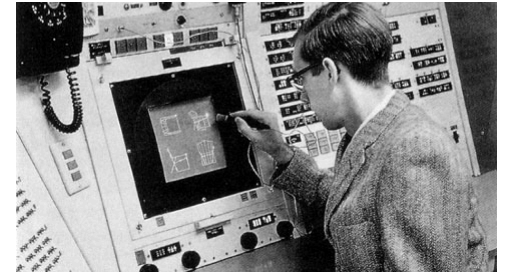


3. ECS (Entity Component System)!

3.1 The history of ECS

3. ECS (Entity Component System)!

- The origins of ECS reach back to 1959, when Ivan Sutherland pioneered a similar idea in a drawing program, one of the first graphical user interfaces (Sutherland, 2003).
- Today, ECS is primarily used in game development, for example in the Bevy engine and Unity DOTS.
- While there is a paper on using ECS for ABMs, it remains a largely unknown programming paradigm in this area (Casals & Brandão, 2025).



3.2 A short introduction

3. ECS (Entity Component System)!

- ECS stands for **Entity Component System**. It is a programming pattern that separates data from behavior.
- **Entities** are unique identifiers. By themselves they do not contain logic or meaning; they simply represent individual objects in the simulation.
- **Components** are small data containers attached to entities. Each component stores one specific aspect of state, such as position, direction, age, or velocity.
- An entity is defined by the set of components it has. For example, a car entity might have components for position, direction, habitus, and step count.

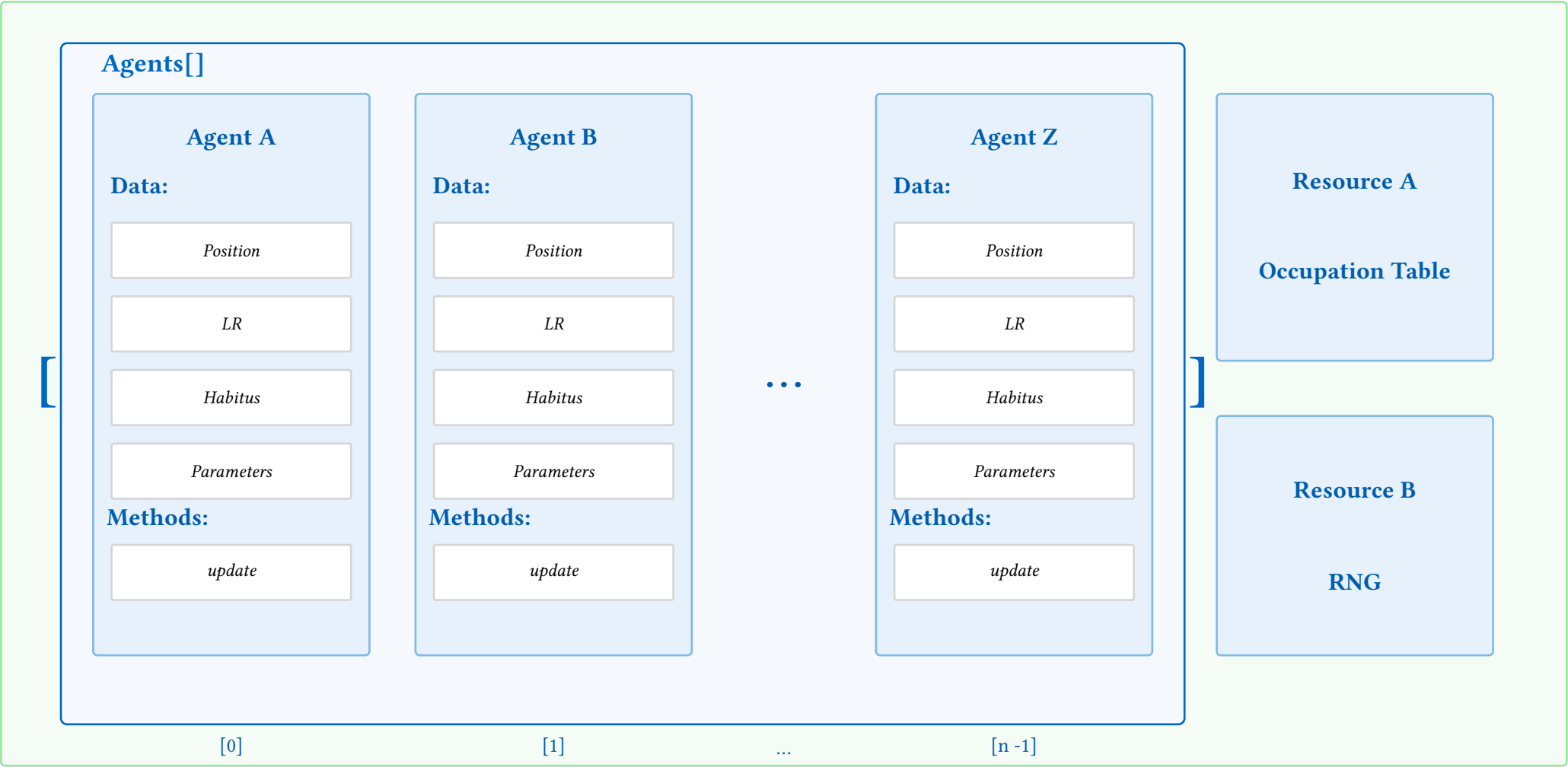
3.2 A short introduction

3. ECS (Entity Component System)!

- **Systems** implement the behavior of the model. They query all entities that have a required set of components and then update those components according to the model rules.
- This means that data and functionality are kept separate: components hold data, while systems contain the logic that acts on that data.

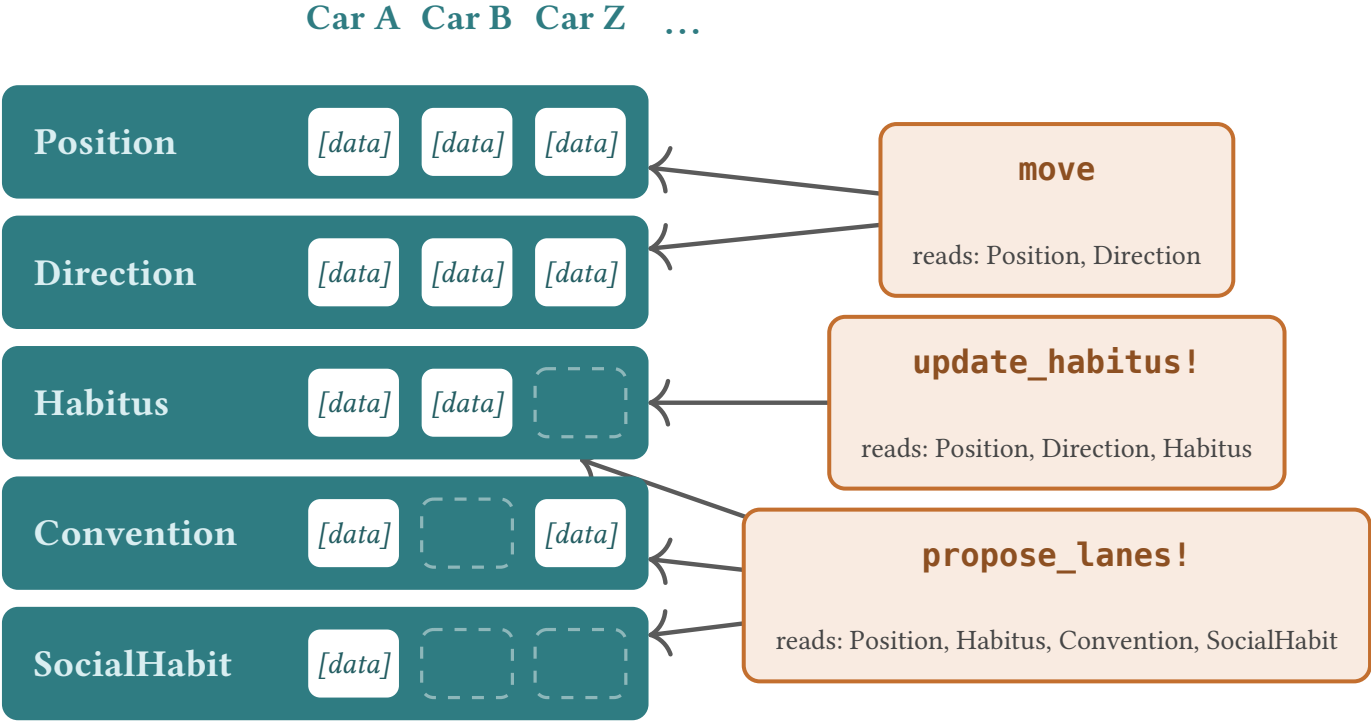
3.3 Data layout in a classic ABM

3. ECS (Entity Component System)!



3.4 ECS data layout

3. ECS (Entity Component System)!



3.5 The habit rule in both layouts

3. ECS (Entity Component System)!

Classic ABM (Agents.jl)

```
@agent struct Car{GridAgent{2}}
    lr::Float64
    habitus::Float64
    direction::Direction
    age::Int64
end

function update_habitus!(agent, model)
    agent.habitus = clamp(
        agent.habitus + relative_lane_sign(
            agent.pos[1], agent.direction
        ) / (model.params.K + agent.age),
        -1.0, 1.0,
    )
end

function agent_step!(agent, model)
    calculate_lr!(agent, model)
    move_agent!(agent, intended_position(agent, model), model)
    agent.age += 1
    update_habitus!(agent, model)
end
```

ECS (Ark.jl)

```
struct Position
    x::Int64
    y::Int64
end

struct Habitus
    val::Float64
end

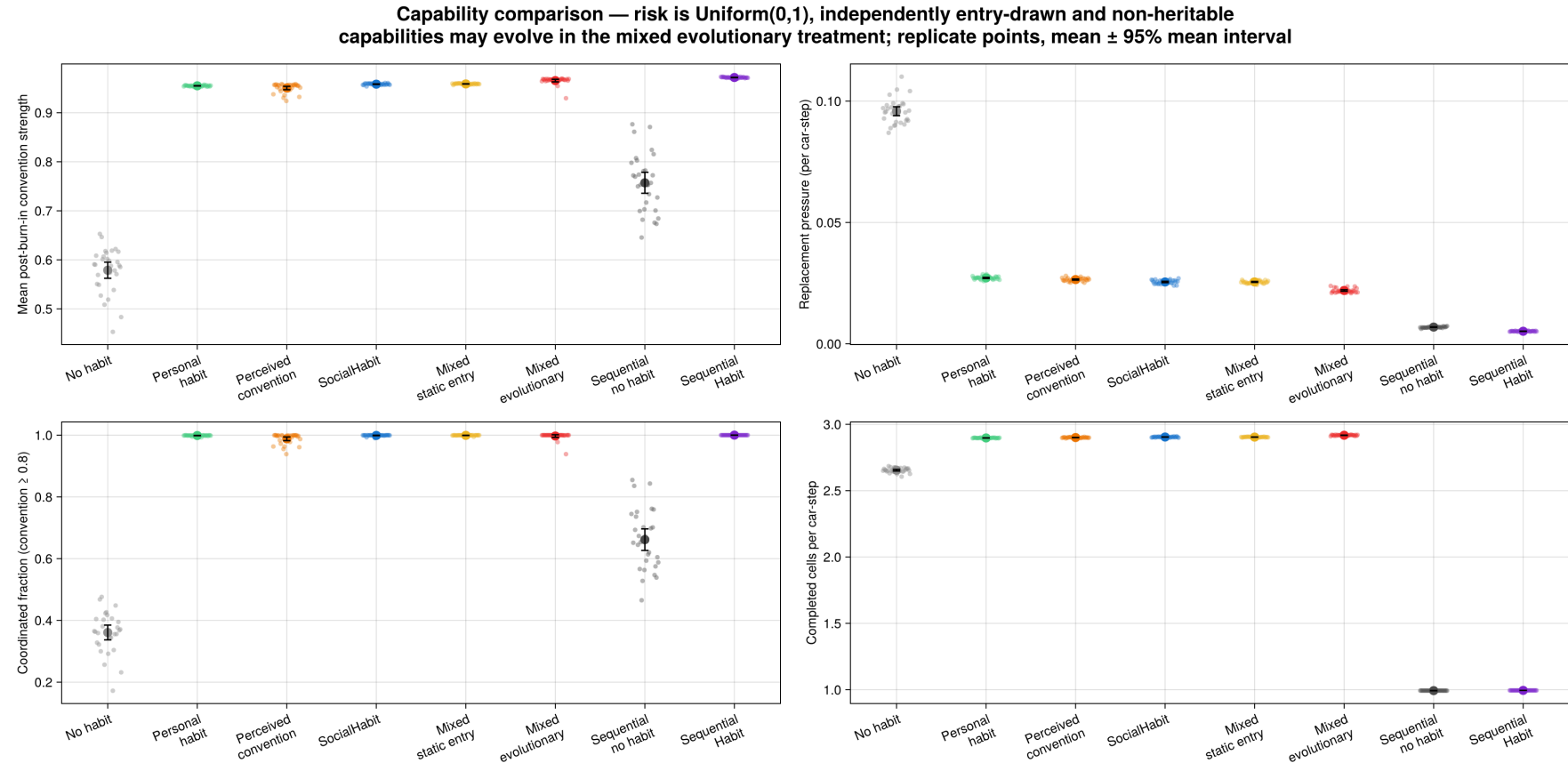
function update_habitus!(world)
    params = Ark.get_resource(world, ModelParams)
    for (e, pos, dir, hab, step) in
        Query(world, (Position, Direction, Habitus, Step))
        @inbounds for i in eachindex(e)
            hab[i] = Habitus(clamp(
                hab[i].val + relative_lane_sign(
                    pos[i].x, dir[i]
                ) / (params.K + step[i].val),
                -1.0, 1.0,
            ))
        end
    end
end
```

4. Current traffic model

- **Habit:** the driver's own realized-side history, reinforced by the age-dependent Hodgson–Knudsen rule.
- **Convention:** a private history of locally observed side choices made by **other** drivers (Ellison, 1993).
- **SocialHabit:** a private history of locally observed, decaying traces left by successful drivers.

LR is an **additive score**: each mechanism adds its own weighted term, $w_h \cdot \text{disposition} \cdot h^n$ (habit), $w_c \cdot \text{confidence} \cdot c^n$ (convention), via a dedicated scoring system; `propose_lanes!` commits the **sign** of the sum.

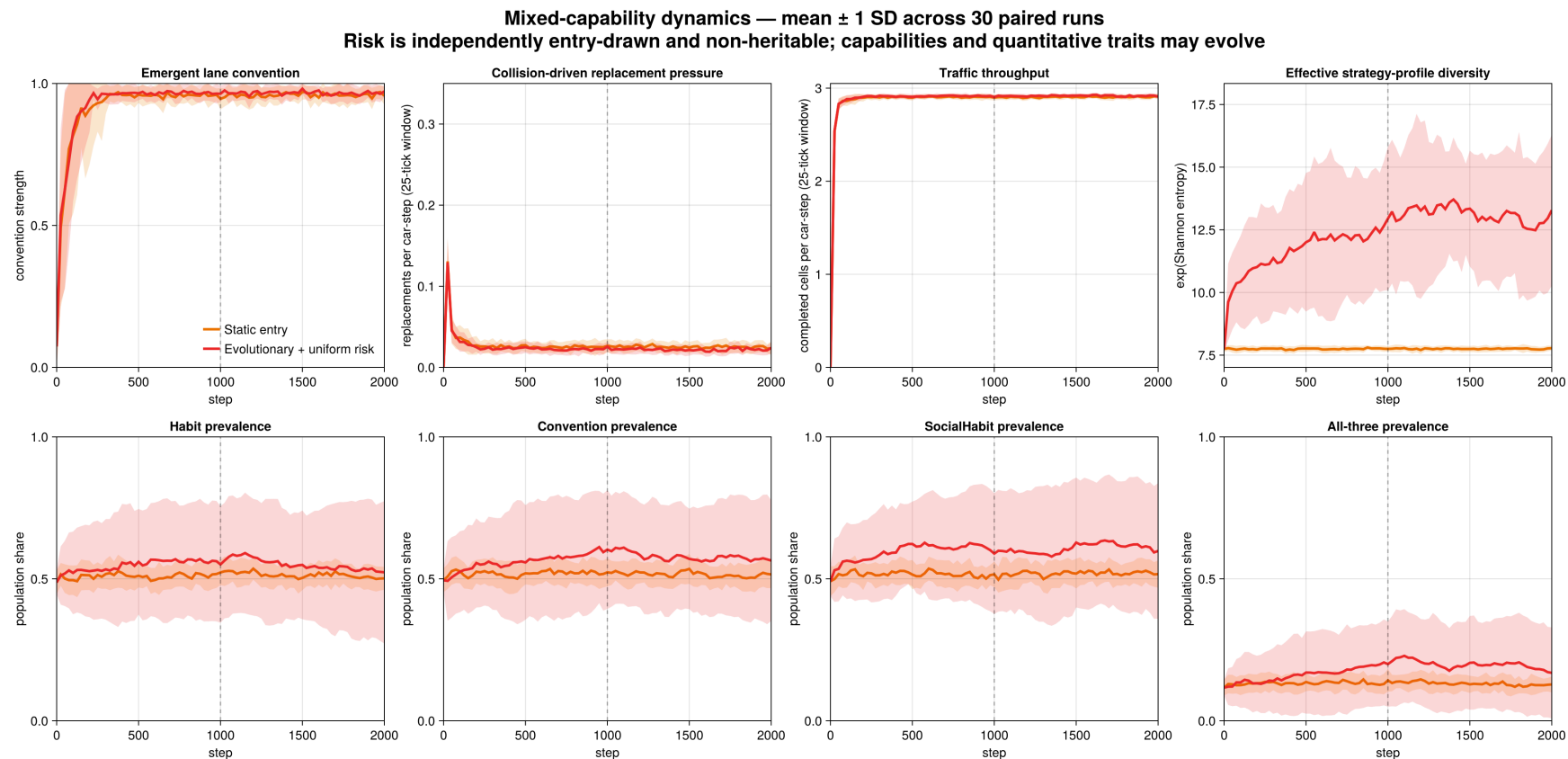
4.2 Capabilities under uniform entry risk 4. Current traffic model



Takeaway: non-heritable entry risk lets the mixed evolutionary treatment isolate capability evolution from inherited-risk selection.

4.3 Mixed capability dynamics

4. Current traffic model



Takeaway: mean capability shares are higher under mixed evolution, but between-seed variation is substantial.

5. Possible benefits and drawbacks of using ECS for ABMs

5.1 Benefits

5. Possible benefits and drawbacks of using ECS for ABMs

- More natural support for **parallel agent interactions**, both conceptually and computationally.
- **Compositional heterogeneity** offers a new approach to heterogeneous agents. (Dynamic acquisition of new capabilities)
- Potential **performance** benefits compared to “agent-oriented” implementations (*JuliaDynamics/ABMFrameworksComparison*, 2026).
 - Easy to utilize the GPU.
 - More ergonomic than the big parallel ABM frameworks (FLAME GPU) with potentially minimal loss in performance.
- Clear support for **modularity** and separation of concerns
- Encourages a more **systemic** rather than purely individual-centered modeling perspective

5.2 Drawbacks

5. Possible benefits and drawbacks of using ECS for ABMs

- **Agent-centric thinking does not map directly onto entity-component tables:** the mental shift is substantial for most modelers.
- **Boilerplate:** every state variable becomes a wrapper type, and behavior is scattered across many small systems.
- Very **limited ABM literature** and tooling: no equivalent of the Agents.jl, Mesa, or NetLogo ecosystems (Casals & Brandão, 2025).
- Performance is framework- and workload-dependent (*JuliaDynamics/ABMFrameworksComparison*, 2026).

5.3 Conclusion

5. Possible benefits and drawbacks of using ECS for ABMs

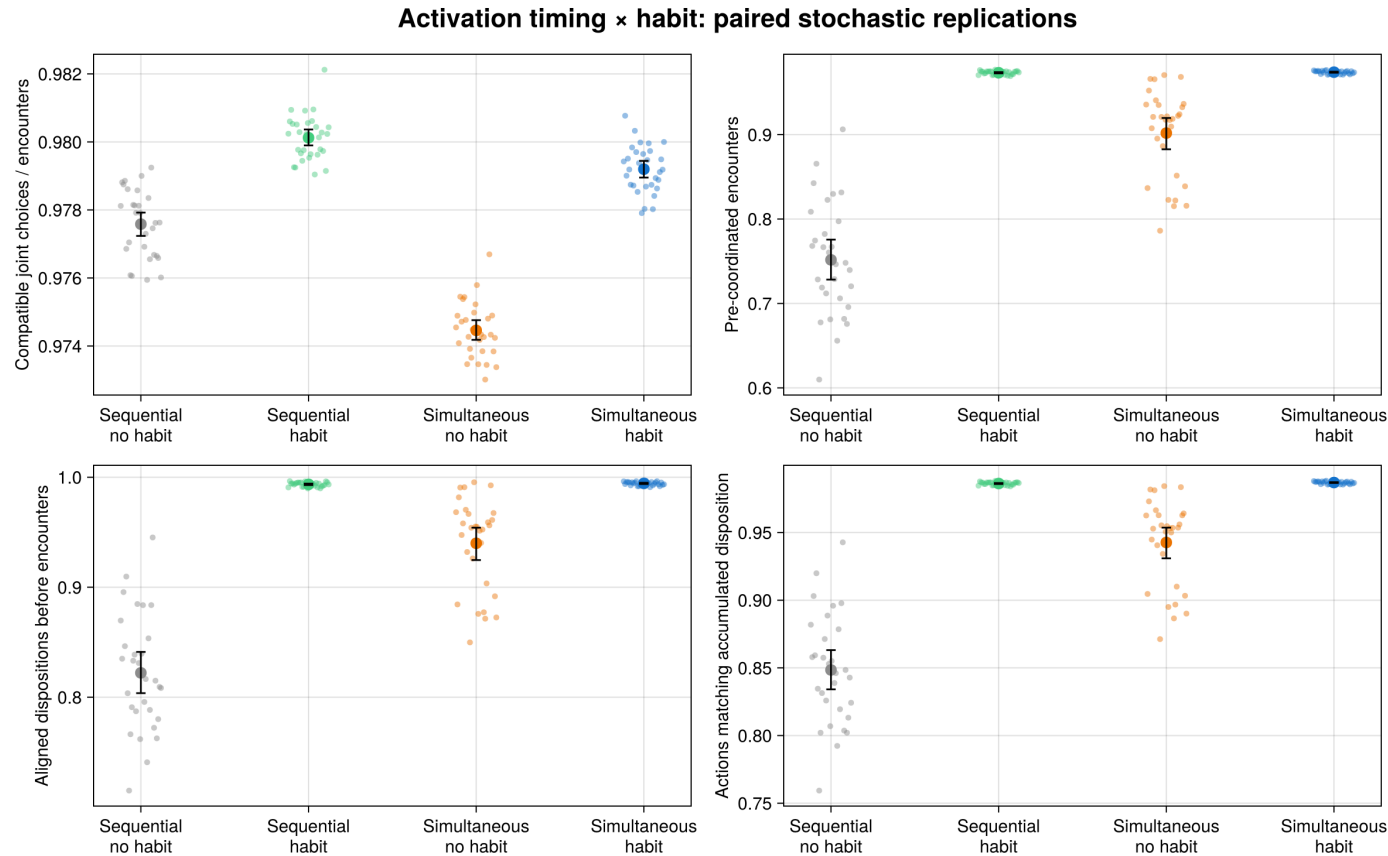
- **The choice of framework is not neutral:** it affects modeling decisions about the information agents hold, update ordering, sequential versus parallel action, and the type of heterogeneity agents possess.
- **ECS offers an alternative to agent-oriented frameworks.**
- The extensions of Hodgson & Knudsen (2006) show how architecture shapes which modeling choices are easiest to express: agent-oriented layouts favor activation-order updates, while ECS makes staged synchronous resolution more explicit
- **ECS is promising for ABMs, but remains underexplored** conceptually and methodologically.

6. Appendix

Concept	Description	ABM Equivalent	Role in Simulation
Entity	Unique object in the world	Agent	Represents an individual actor in the simulation
Component	Data attached to an entity	Agent attributes / state variables	Stores properties such as position, preferences, or resources
System	Function operating on sets of components	Part of the agent step function	Implements simulation rules and updates state

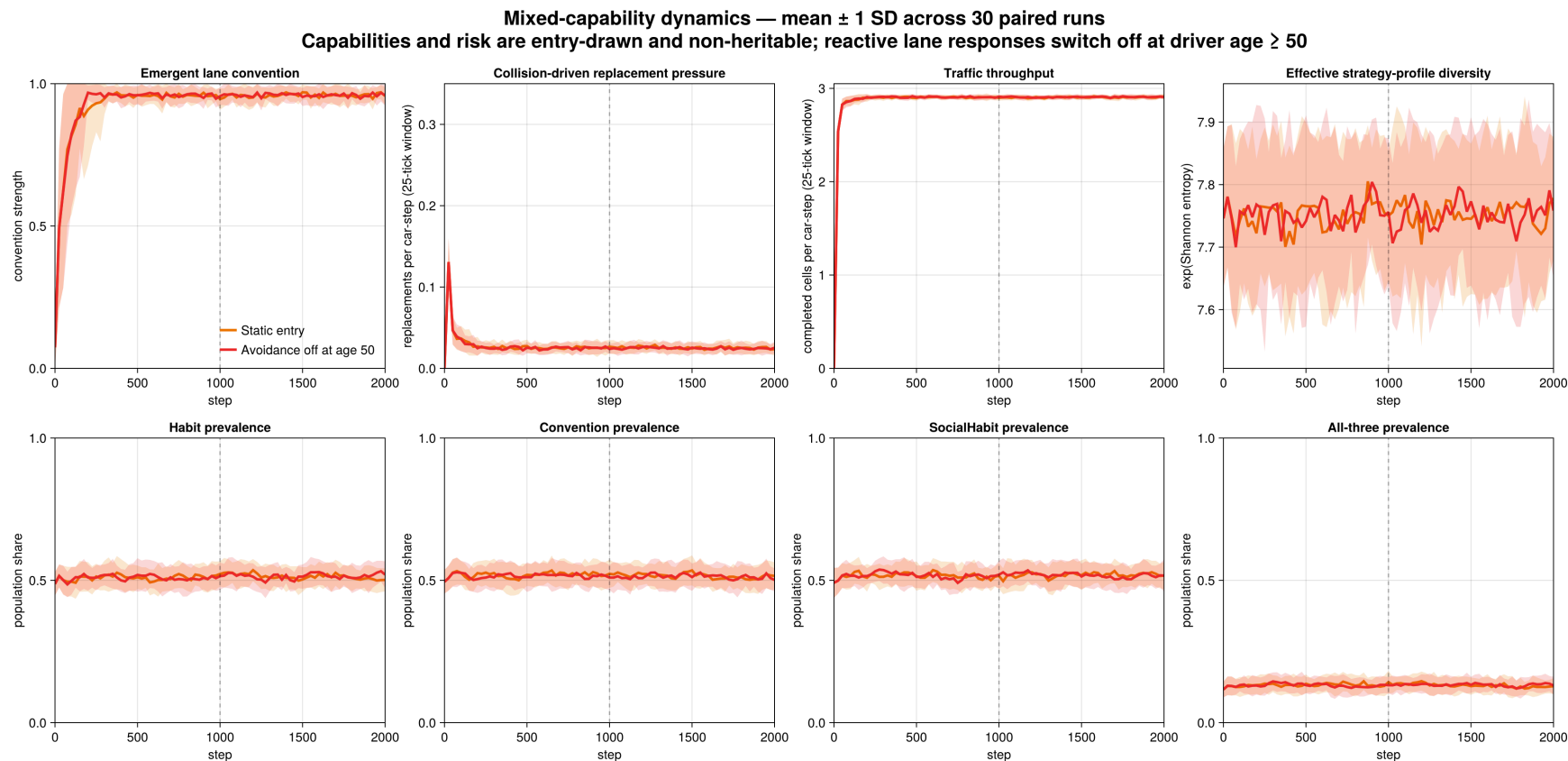
6.2 New additions

- All cars decide from **one committed pre-decision state**.
- LR determines the final lane; speed is chosen only on that binding lane.
- A car attempts maximum speed (normally 3), makes one seeded draw, and accepts danger with probability $1 - \text{risk aversion}$.
- Rejected danger causes progressively lower-speed tests; speed 1 is the unavoidable-risk fallback when every speed is dangerous.



Takeaway: timing changes compatibility; habit more than offsets the simultaneous loss.

6.4 Avoidance retires at age 50



Takeaway: habit, convention, and social habit sustain the convention after the reactive responses switch off, with only small aggregate changes.

6.5 Outlook

- Scheduler (Helm.jl) for Ark.jl in progress.
 - Helps make dependencies of systems explicit, leading to automatic parallel scheduling between systems and easier modularity.
 - Could enable automatic ODD exporting (Grimm et al., 2020).
- Rewrote BeforeIT (Glielmo et al., 2025; Poledna et al., 2023) from a Structure of Arrays approach to ECS
 - Modular Macro-ABM?
- More experiments with ABMs that truly benefit from compositional heterogeneity.

6.6 Code: LR emerges from system composition

6. Appendix

```
struct ReactiveLaneResponse end

function add_same_direction_response!(world)
    weight = Ark.get_resource(world, Weights).w_s
    for (_, observations, responses, scores, steps) in Query(
        world, (LocalObservation, SameDirectionResponse,
                LaneScore, Step);
        with = (ReactiveLaneResponse,),
    )
        for i in eachindex(scores)
            value = weight * responses[i].sensitivity *
                    (2 * observations[i].same_left - 1)
            scores[i] = LaneScore(scores[i].value + value)
        end
    end
end

function add_habit_response!(world)
    model = Ark.get_resource(world, CapabilityModel)
    for (_, habits, habitus, scores) in Query(
        world, (HabitFormation, Habitus, LaneScore),
    )
        for i in eachindex(scores)
            value = model.habit_weight *
                    habits[i].disposition * habitus[i].value
            scores[i] = LaneScore(scores[i].value + value)
        end
    end
end

function add_convention_response!(world)
    model = Ark.get_resource(world, CapabilityModel)
    for (_, perceptions, scores) in Query(
        world, (PerceivedConvention, LaneScore),
    )
        for i in eachindex(scores)
            p = perceptions[i]
            value = model.convention_weight * p.confidence * p.value
            scores[i] = LaneScore(scores[i].value + value)
        end
    end
end
```

```
function calculate_capability_proposals!(world)
    observe_capability_traffic!(world)
    learn_conventions!(world)
    learn_social_habits!(world)
    reset_lane_scores!(world)
    add_same_direction_response!(world)
    add_opposite_direction_response!(world)
    add_near_field_avoidance!(world)
    add_habit_response!(world)
    add_convention_response!(world)
    add_social_habit_response!(world)
    propose_lanes!(world)
end

function propose_lanes!(world)
    for (_, positions, directions, scores,
        proposals, decisions) in Query(
        world, (Position, Direction, LaneScore,
                LaneProposal, LR),
    )
        for i in eachindex(scores)
            score = scores[i].value
            # The sign selects the relative lane.
            proposals[i] = LaneProposal(...)
            decisions[i] = LR(score)
        end
    end
end

# One structural switch disables all three
# reactive scoring systems for this car:
Ark.remove_components!(
    world, car, (ReactiveLaneResponse,),
)
```

Each system contributes one term to the same LaneScore; the final system commits its sum as LR.

6.7 Code: spawning a capability-composed car

```
function capability_components(genome)
  components = ()

  !isnothing(genome.same_direction) &&
  (components = (
    components...,
    SameDirectionResponse(
      genome.same_direction,
    ),
  ))

  if !isnothing(genome.habit)
    components = (
      components...,
      HabitFormation(genome.habit),
      Habitus(0.0),
    )
  end

  # Other optional capabilities follow
  # the same composition pattern.
  return components
end
```

```
function spawn_capability_car!(
  world, position, direction, speed, genome, model,
)
  base = (
    position,
    PrevPosition(position),
    direction,
    ReactiveLaneResponse(),
    RiskAversion(genome.risk_aversion),
    Speed(speed),
    SpeedAdjustment(model.max_speed),
    LocalObservation(),
    LaneScore(0.0),
    LaneProposal(position.x),
    SpeedProposal(speed),
    MovementPath(position),
    LR(0.0),
    Step(1),
  )
  return Ark.new_entity!(
    world,
    (base..., capability_components(genome)...),
  )
end
```

A car is an entity assembled from mandatory state plus only the behavioral capabilities present in its genome.

6.8 Code: selecting an evolutionary parent

```
function spawn_new_entities!(world, replacements)
    model = Ark.get_resource(world, CapabilityModel)

    survivor_data = []
    for (entities, positions) in Query(world, (Position,))
        for i in eachindex(entities)
            push!(survivor_data, (
                entities[i],
                positions[i],
                capability_genome(world, entities[i]),
            ))
        end
    end

    sort!(survivor_data;
        by = survivor -> entity_order(survivor[1]))
    parent_genomes = [s[3] for s in survivor_data]

    # For every replacement:
    genome = replacement_genome(
        model.replacement_policy,
        parent_genomes,
        model, draws, rng,
    )
    spawn_capability_car!(
        world, position, direction,
        model.max_speed, genome, model,
    )
end
```

```
function replacement_genome(
    ::EntryDrawReplacement,
    parent_genomes, model, draws, rng,
)
    return entry_capability_genome(
        model, draws, rng, rand(rng),
    )
end

function replacement_genome(
    policy::EvolutionaryReplacement,
    parent_genomes, model, draws, rng,
)
    isempty(parent_genomes) &&
        return entry_capability_genome(model, draws, rng)

    parent = rand(rng, parent_genomes)
    return inherit_capability_genome(
        parent, model, draws, policy, rng,
    )
end
```

Collided cars are already gone, so the query contains only survivors. Evolution samples one of their genomes uniformly.

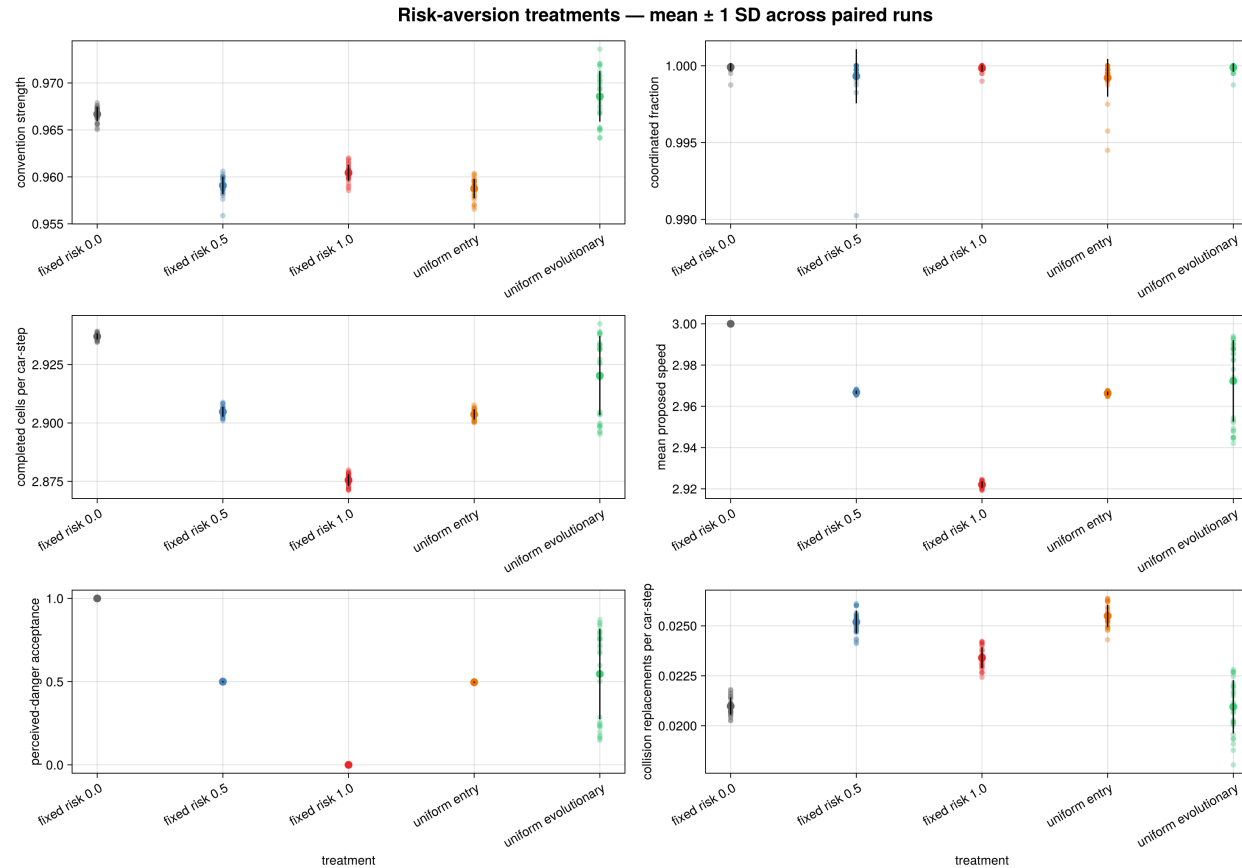
```
function mutate_optional_trait(  
  parent, entry_value, enabled, policy, rng,  
)  
  if rand(rng) < policy.capability_mutation_rate  
    return isnothing(parent) ?  
      (enabled ? entry_value : nothing) :  
      nothing  
  end  
  
  isnothing(parent) && return nothing  
  return max(  
    0.0,  
    parent + policy.trait_mutation_scale * randn(rng),  
  )  
end
```

A mutation event flips capability presence. Otherwise its quantitative trait is inherited with Gaussian mutation.

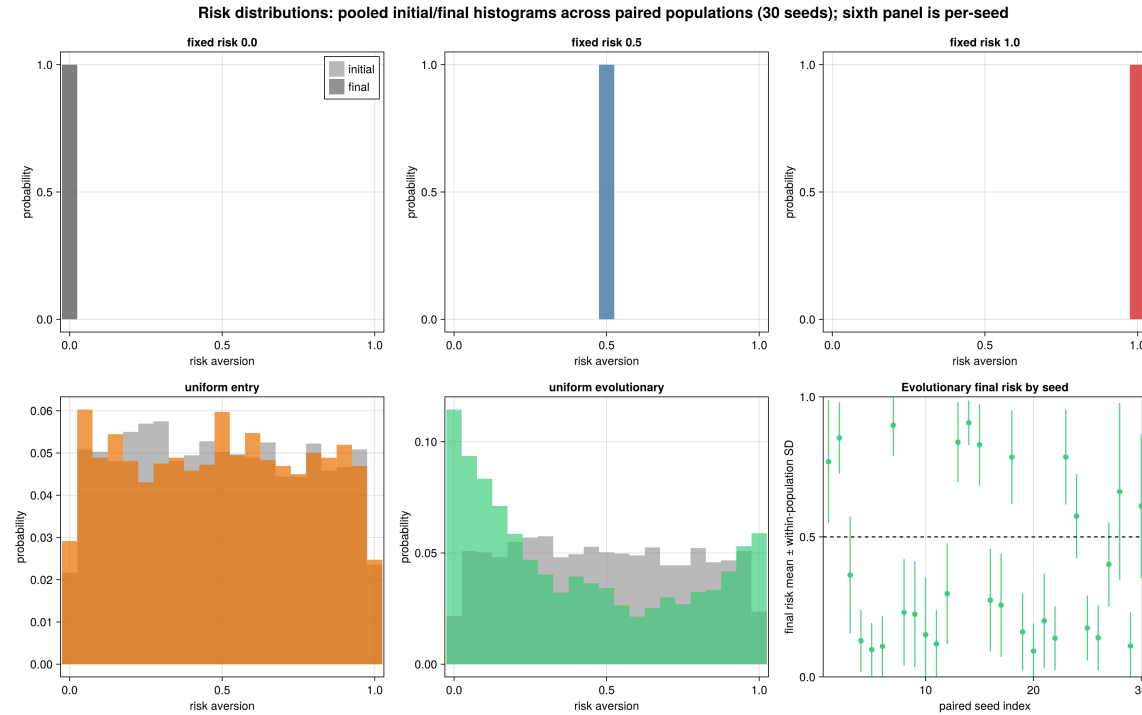
```
function inherit_capability_genome(  
  parent, model, draws,  
  policy::EvolutionaryReplacement, rng,  
)  
  return CapabilityGenome(  
    clamp(  
      parent.risk_aversion +  
      policy.trait_mutation_scale * randn(rng),  
      0.0, 1.0,  
    ),  
    mutate_optional_trait(  
      parent.same_direction, draws[1],  
      model.same_direction_share > 0.0,  
      policy, rng,  
    ),  
    # opposite direction, avoidance, habit,  
    # convention, and social habit follow.  
  )  
end
```

The genome contains stable capabilities and traits only; acquired dispositions are reset when the child is spawned.

6.10 Robustness: fixed and evolving risk aversion 6. Appendix



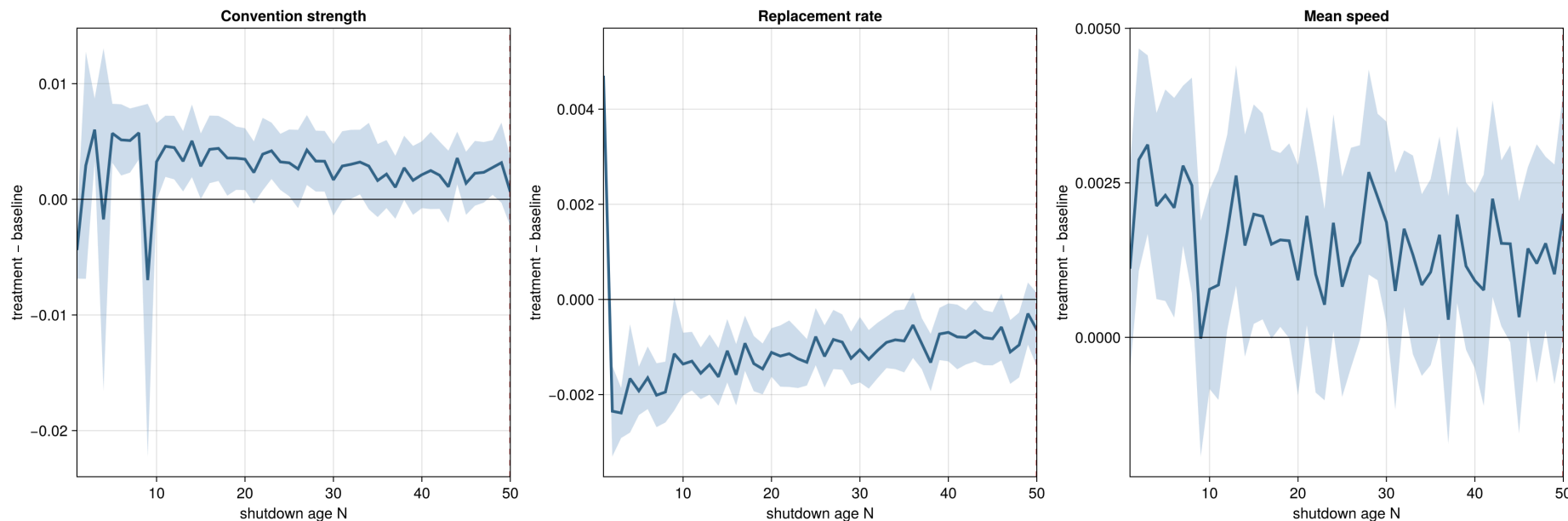
Takeaway: fixed zero risk has the highest throughput, while replacement pressure is nonmonotone.



Takeaway: evolution improves average outcomes, but survival selection is path dependent.

6.12 Robustness: avoidance shutdown age

Avoidance shutdown-age robustness — 30 paired runs, post-burn metrics
Differences are treatment minus the same-seed no-shutdown baseline; ribbon is a 95% normal paired-mean CI



Takeaway: most thresholds slightly strengthen convention and lower replacement pressure; age 1 produces the clearest reversal, while speed effects remain small.

Bibliography

- [1] G. M. Hodgson and T. Knudsen, *Economics in the Shadows of Darwin and Marx: Essays on Institutional and Evolutionary Themes*. Edward Elgar Publishing, 2006. doi: 10.4337/9781781007563.
- [2] B. A. Huberman and N. S. Glance, “Evolutionary Games and Computer Simulations,” *Proceedings of the National Academy of Sciences*, vol. 90, no. 16, pp. 7716–7718, Aug. 1993, doi: 10.1073/pnas.90.16.7716.
- [3] I. E. Sutherland, “Sketchpad: A Man-Machine Graphical Communication System,” 2003.

- [4] A. Casals and A. A. F. Brandão, “HECATE: An ECS-based Framework for Teaching and Developing Multi-Agent Systems.” Accessed: Feb. 02, 2026. [Online]. Available: <http://arxiv.org/abs/2509.06431>
- [5] G. Ellison, “Learning, Local Interaction, and Coordination,” *Econometrica*, vol. 61, no. 5, p. 1047, Sep. 1993, doi: 10.2307/2951493.
- [6] “JuliaDynamics/ABMFrameworksComparison.” Accessed: Apr. 06, 2026. [Online]. Available: <https://github.com/JuliaDynamics/ABMFrameworksComparison>
- [7] V. Grimm *et al.*, “The ODD Protocol for Describing Agent-Based and Other Simulation Models: A Second Update to Improve Clarity,

- Replication, and Structural Realism,” *Journal of Artificial Societies and Social Simulation*, vol. 23, no. 2, p. 7, 2020, doi: 10.18564/jasss.4259.
- [8] A. Glielmo, M. Devetak, A. Meligrana, and S. Poledna, “BeforeIT.Jl: High-Performance Agent-Based Macroeconomics Made Easy.” Accessed: Sep. 04, 2026. [Online]. Available: <http://arxiv.org/abs/2502.13267>
- [9] S. Poledna, M. G. Miess, C. Hommes, and K. Rabitsch, “Economic Forecasting with an Agent-Based Model,” *European Economic Review*, vol. 151, p. 104306, Jan. 2023, doi: 10.1016/j.eurocorev.2022.104306.